Implementing a Serverless Calculator Function Using OpenFaaS

Introduction

For this task I built and deployed a simple calculator function using OpenFaaS on a local Kubernetes cluster. I went with a calculator rather than a greeting service mainly because it gave me a reason to include some input validation and error handling, which felt closer to what a real function would need to do. Toosi et al. (2025) describe serverless computing as a model where developers write and deploy individual functions while the platform takes care of provisioning, scaling and running the infrastructure underneath, and that was the part I most wanted to see for myself, how much of the operational work actually disappears once the platform is set up.

Setting Up the Environment

I set everything up on my Mac using kind, which runs a Kubernetes cluster inside Docker. I installed kubectl, kind and faas-cli through arkade, then created a single node cluster called openfaas-lab. Figure 1 shows the node in a Ready state, confirming the cluster was available to host OpenFaaS.

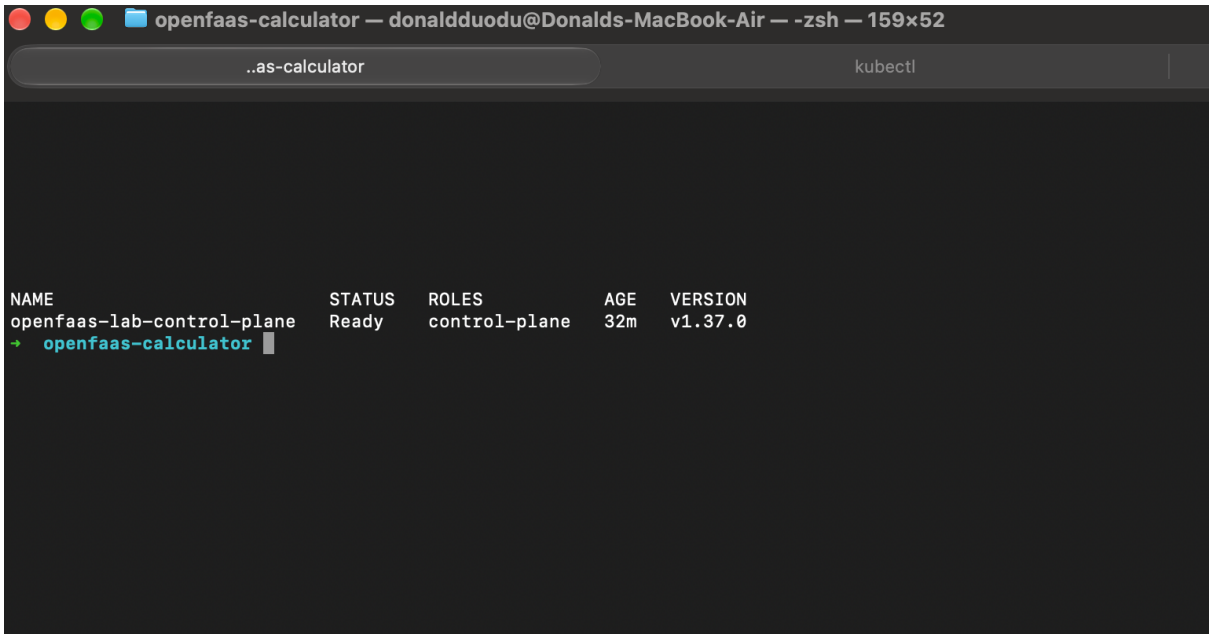


Figure 1: Kubernetes node in Ready state

Getting to this point took longer than I expected. The tools installed fine but my terminal could not find them until I added the arkade folder to my PATH, and the cluster then failed to create because Docker was not installed yet. Neither was difficult to fix, but it was a reminder that a lot of the effort in getting started sits in the local tooling rather than the platform itself.

Installing OpenFaaS

The main issue I hit was with the OpenFaaS install. Running arkade install openfaas installed OpenFaaS Pro by default rather than the free Community Edition, and the gateway sat waiting for its replicas without ever starting, which I eventually traced to Pro needing a licence key. I removed it with Helm and reinstalled the Community Edition directly from the OpenFaaS Helm chart instead (OpenFaaS, n.d.a). I think this was actually a useful thing to run into, since it shows how a default in an open source tool can change over time and quietly push users towards a paid tier, which links back to the point Dunbar (2024) makes about dependency on a particular vendor's tooling building up without it being obvious at the time. Figure 2 shows the Community Edition components running, including the gateway, NATS, the queue worker, Prometheus and alertmanager.

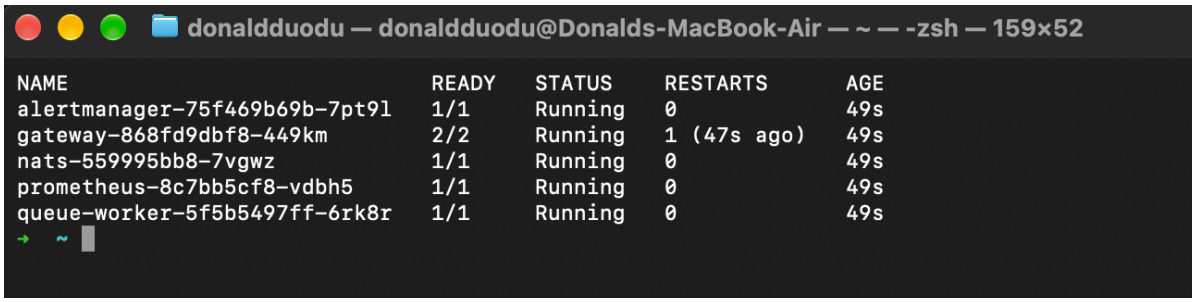


Figure 2: OpenFaaS Community Edition components running

I then port forwarded the gateway to my machine and logged in with faas-cli using the generated admin password, shown in Figure 3. The gateway is the single entry point for every function call, so this is the part of OpenFaaS that clients actually talk to rather than individual pods.

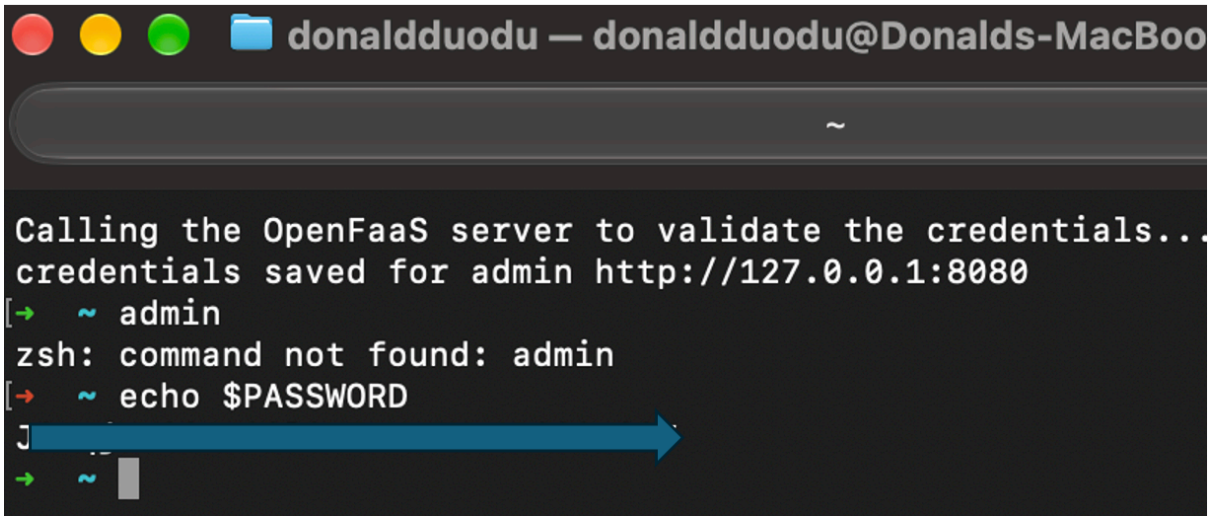


Figure 3: Logging in to the OpenFaaS gateway (Password is hidden)

Building the Calculator Function

I created the function from the python3-http template using faas-cli. The function reads a JSON body containing two numbers and an operation, carries out the calculation, and returns the result. I added validation so that missing or non numeric inputs, an unsupported operation or a divide by zero all return a 400 response

```
import json

def handle(event, context):
    # Read the JSON body and check the inputs are valid
    try:
        data = json.loads(event.body)
        a = float(data["a"])
        b = float(data["b"])
        operation = data["op"]
    except (ValueError, KeyError, TypeError):
        return {
            "statusCode": 400,
            "body": json.dumps({"error": "Send JSON like {\"a\": 6, \"b\": 3, \"op\": \"add\"}"})
        }

    if operation == "add":
        result = a + b
    elif operation == "subtract":
        result = a - b
    elif operation == "multiply":
        result = a * b
    elif operation == "divide":
        if b == 0:
            return {
                "statusCode": 400,
                "body": json.dumps({"error": "Cannot divide by zero"})
            }
        result = a / b
    else:
        return {
            "statusCode": 400,
            "body": json.dumps({"error": "Operation must be add, subtract, multiply or divide"})
        }

    return {
        "statusCode": 200,
        "body": json.dumps({"result": result})
    }
```

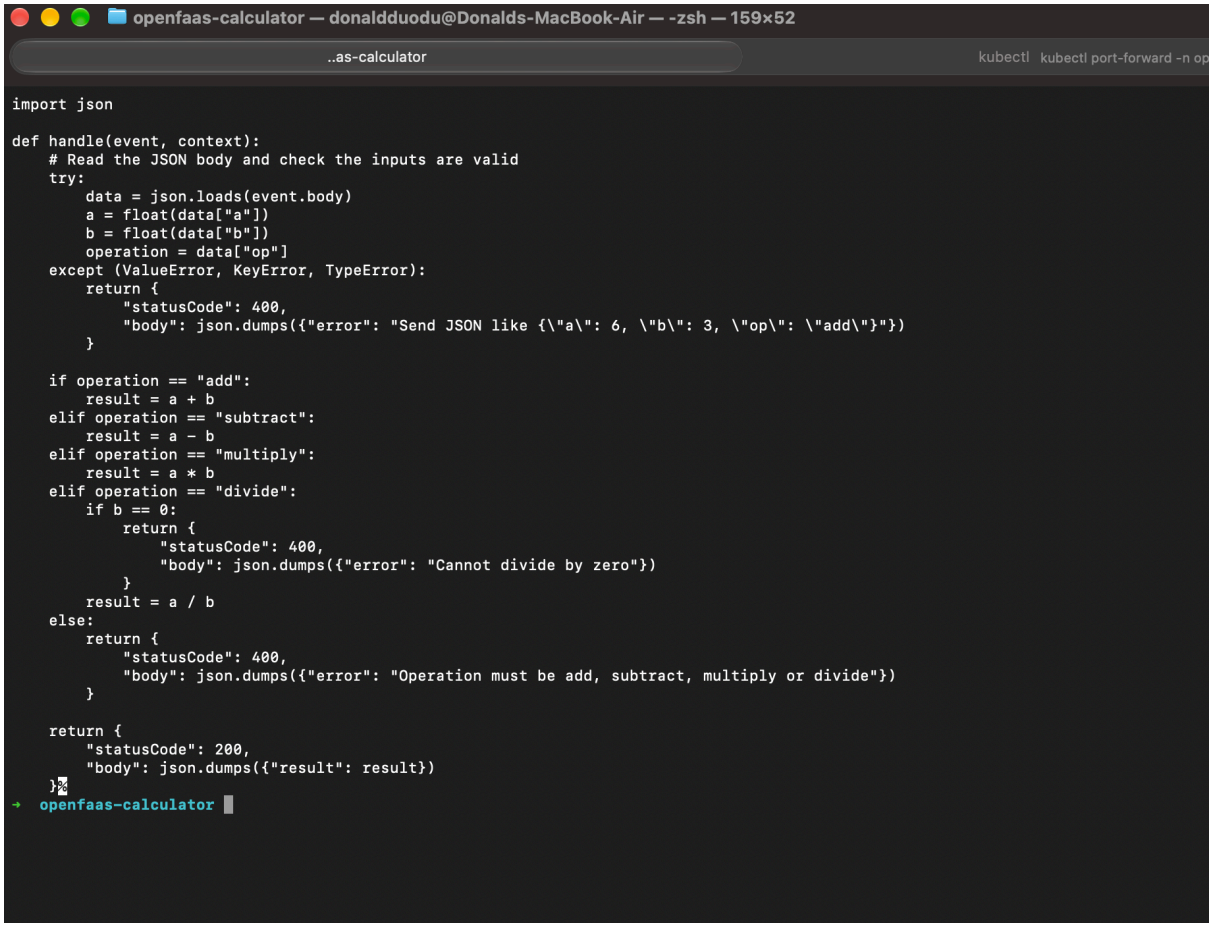


Figure 4: Calculator function handler code

Figure 5 shows the stack.yaml file, which tells faas-cli which gateway to deploy to, which template to use, where the handler code sits and what to call the container image (OpenFaaS, n.d.b). My first build actually failed here because I had left a placeholder in the image name instead of my Docker Hub username, which Docker rejected since image names have to be lowercase.

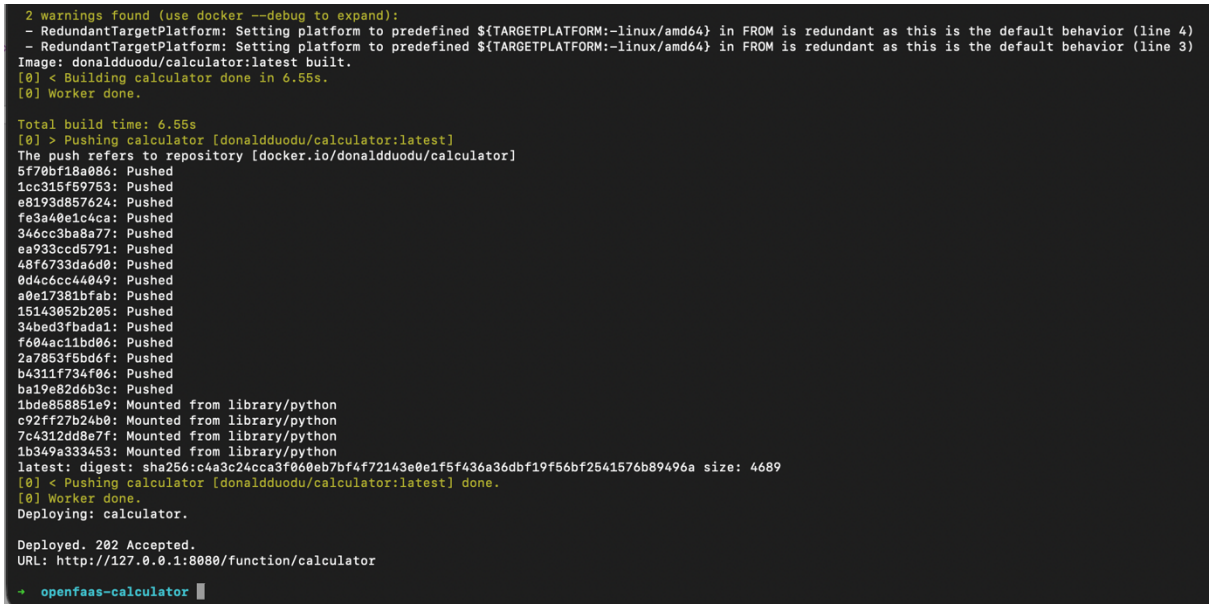


Figure 5: OpenFaaS stack.yaml configuration

Deploying and Invoking the Function

Once the image name was fixed, I ran faas-cli up, which built the container image, pushed it to Docker Hub and deployed it to the cluster in one step. Figure 6 shows the 202 Accepted response and the function URL, along with faas-cli list confirming the function was registered.

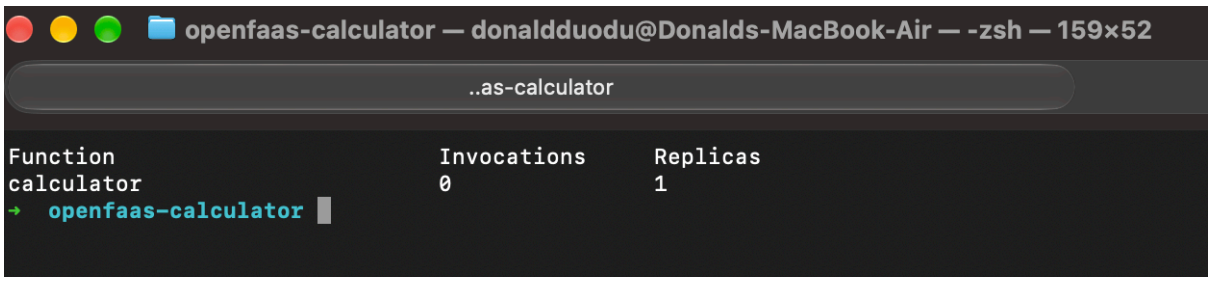


Figure 6: Successful deployment of the calculator function

I then invoked the function with a few different inputs. Figure 7 shows a multiplication and a subtraction returning the correct results, and the divide by zero request returning the error message instead of failing.

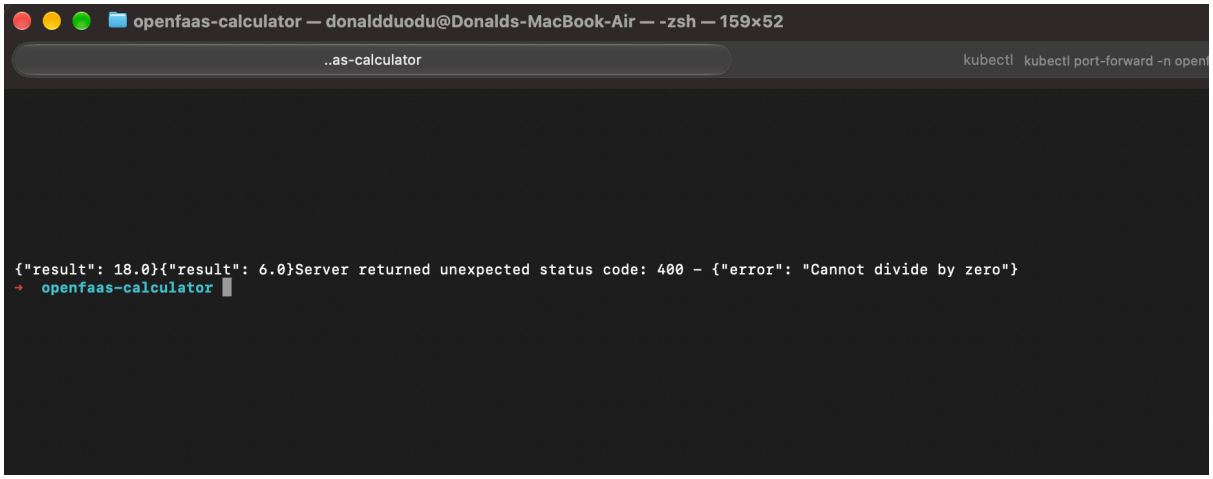


Figure 7: Invoking the calculator function, including error handling

How Serverless Optimises Cloud Operations

To see the operational side, I scaled the function from one replica to three. Figure 8 shows this happening without rebuilding or redeploying anything, the platform simply started more copies of the same function behind the gateway.

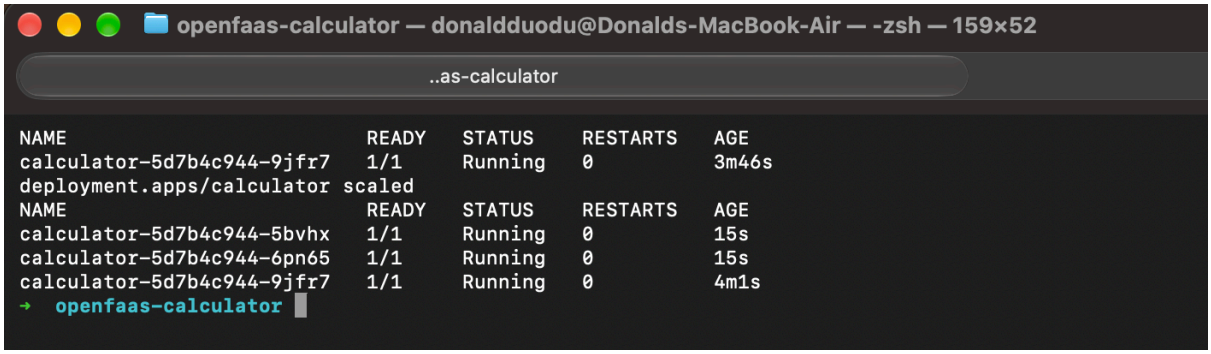


Figure 8: Scaling the calculator function from one to three replicas

I should be honest that this was done manually. Automatic scaling based on load and scaling down to zero are part of the paid OpenFaaS editions, so with the Community Edition I could show that scaling works but not that it happens on its own. Even so, the benefit was clear. The function code knows nothing about how many copies of it are running, which is the separation between application logic and infrastructure that Toosi et al. (2025) highlight as one of the main operational advantages of serverless. It also connects to the cost argument. Ahmad et al. (2023) look at reducing cloud costs by matching resources more closely to the workload actually being run, and serverless pushes that idea further, since in a platform with scale to zero you only pay for compute while a function is actually doing something rather than for a server sitting idle.

Finally, I sent a burst of requests to the function and checked the built in Prometheus metrics. Figure 9 shows invocation counts for the calculator broken down by status code, including the 400 responses from my error tests. Getting this level of monitoring without writing any monitoring code myself was one of the more useful things I noticed, although Toosi et al. (2025) also point out that observability and debugging across many small functions can become harder as systems grow, which I can see would be true once there were dozens of functions rather than one.

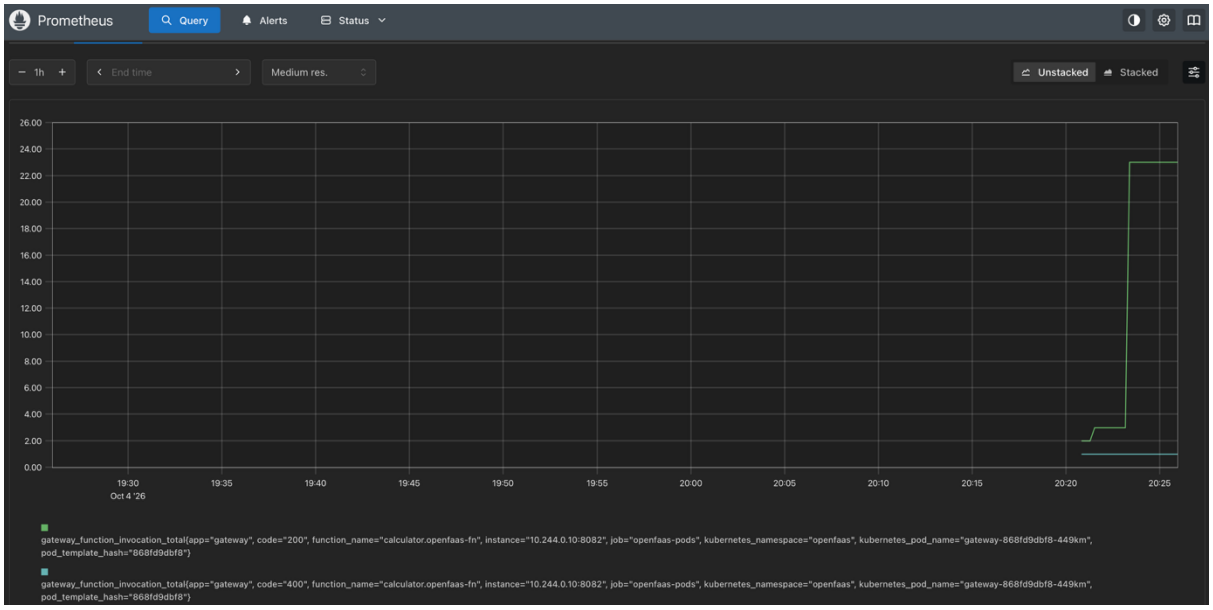


Figure 9: Prometheus metrics showing calculator invocations by status code

Conclusion

Overall, the deployment showed that once OpenFaaS is in place, getting a function from code to a working, scalable HTTP endpoint is quick and mostly hands off. The harder parts for me were the setup around it, the local tooling, Docker and the Pro versus Community Edition issue, rather than the function itself. The main limitation of my implementation is that scaling was manual, so I would want to try a paid edition or a managed serverless platform to properly test automatic scaling and cold start behaviour, which is where the real cost and performance benefits of serverless would show up.

References

Ahmad, S.G. et al. (2023) 'Cost optimization in cloud environment based on task deadline', *Journal of Cloud Computing*, 12(1), p. 9.

Dunbar, W. (2024) *Vendor-Locked DevOps Strategies*. New York: Association for Computing Machinery.

OpenFaaS (n.d.a) *Deploy OpenFaaS to Kubernetes*. Available at: https://docs.openfaas.com/deployment/kubernetes/(Accessed: 4 October 2026).

OpenFaaS (n.d.b) *YAML reference*. Available at: https://docs.openfaas.com/reference/yaml/ (Accessed: 4 October 2026).

Toosi, A.N. et al. (2025) 'Serverless computing for next-generation application development', *Future Generation Computer Systems*, 164, p. 107573.